

Lista de Problemas

FacilitaLab — Revisão Técnica

25 itens em 3 categorias de prioridade

Resumo por Prioridade

Símbolo	Categoria	Itens	Impacto
●	CRÍTICO — Quebram build, segurança ou geram comportamento errado	5	Imediato
●	IMPORTANTE — Funcionam, mas com falha lógica ou de segurança	6	Próximo sprint
●	MELHORIA FUTURA — Sem impacto para o protótipo	14	Planejado

● Críticos — Quebram build, segurança ou geram comportamento errado

#	Problema	Descrição / Impacto
1	Build quebrado — dependências de teste inexistentes	pom.xml:76-98 declara 5 starters spring-boot-starter-*-test que não existem no Spring Boot 4. O correto é um único spring-boot-starter-test. Impede compilar/testar.
2	Segredo vazado no histórico do Git	Senha de app do Gmail e usuário em application-dev.properties:7-8, presentes em 3 commits. .gitignore adicionado depois — arquivo ainda rastreado. Revogar a senha no Google, git rm --cached e limpar o histórico.
3	JwtFilter registrado em duplicidade	JwtFilter tem @Component e é instanciado como @Bean + addFilterBefore no SecurityConfig. Resulta em execução dupla por requisição, uma delas fora da ordenação de segurança. Remover @Component ou desabilitar o auto-registro.
4	PedidoController sem tratamento de exceção	Sem @ExceptionHandler nem @RestControllerAdvice global. Erros do PedidoService ('Pedido não encontrado', 'Dentista não encontrado') retornam HTTP 500 em vez de 404/400.
5	POST /pedidos e PUT /pedidos/{id} sem @Valid	PedidoController.java:37,89. Validações do PedidoCreateDTO (@NotNull em material, tipoProtese, prioridade, prazoEntrega) são silenciosamente ignoradas. Corpo incompleto gera NPE/erro de persistência -> 500.

● Importantes — Funcionam, mas com falha lógica ou de segurança

#	Problema	Descrição / Impacto
6	Autorização por papel não aplicada	SecurityConfig.java:37 só exige authenticated(). Qualquer usuário logado pode deletar/editar qualquer usuário ou pedido. As roles existem no token mas nunca são checadas via @PreAuthorize/hasRole.
7	RedefinirSenhaRequestDTO.novaSenha sem @Size	Permite redefinir senha para '1', enquanto o cadastro exige 6–100 caracteres (UsuarioCreateDTO.java:32). Inconsistência grave de regra de negócio.
8	URL hardcoded no e-mail de recuperação	RecuperacaoSenhaService.java:38 monta http://localhost:8081/redefinir-senha?token=... fixo. Quebra em produção — externalizar para application.properties.
9	Lógica morta no reset de senha	RecuperacaoSenhaService.java:56-67: faz setUsado(true) e em seguida delete(...). O token é sempre deletado, então a flag usado e a checagem 'Token já utilizado' nunca disparam.
10	tokenValido engole Exception genérica	JwtUtil.java:58 captura qualquer Exception e retorna false, ofuscando erros legítimos. Capturar JwtException/ExpiredJwtException especificamente.
11	JwtFilter consulta o banco a cada requisição	JwtFilter.java:45 chama buscarEntidadePorEmail em toda request autenticada, sendo que id e perfil já estão disponíveis no próprio token.

● Melhoria Futura — Sem impacto para o protótipo

#	Problema	Descrição / Impacto
12	Exceções genéricas em toda a camada de serviço	RuntimeException genérica em vez de exceções próprias (ResourceNotFoundException, etc.) dificulta tratamento diferenciado nos controllers.
13	Inconsistência em coleções e método converter() repetido	PedidoService mistura Collectors.toList() e .toList(). Método converter() repetido em 8 chamadas — candidato a MapStruct/ModelMapper.
14	Validação de CRO duplicada no UsuarioService	Mesma lógica repetida em criar() (linhas 40-42) e atualizar() (linhas 112-114). Extrair para método privado reutilizável.
15	cadistald em PedidoCreateDTO nunca é usado	Campo declarado em PedidoCreateDTO.java:45 mas nunca consumido em criarPedido — campo morto que confunde o contrato da API.
16	Enums de Pedido sem nullable = false	O service sempre preenche os valores, mas o banco aceitaria null. Adicionar nullable = false para garantir integridade em nível de schema.
17	ddl-auto=update e include-message=always em configurações	Aceitáveis em dev, perigosos se chegarem a produção sem revisão do profile.
18	Dependência circular contornada com @Lazy	SecurityConfig.java:19 usa @Lazy como solução de contorno — indício de problema arquitetural que deve ser resolvido na estrutura

#	Problema	Descrição / Impacto
		de dependências.
19	Metadados vazios no pom.xml e imports com wildcard	<name>, <description>, <scm> e outros metadados ausentes. Imports com wildcard em vários arquivos prejudicam legibilidade e análise estática.
20	Cobertura de testes 0%	Único teste existente é contextLoads() vazio. Sem testes de unidade, integração ou segurança.
21	Sem paginação nas listagens	Endpoints /usuarios, /pedidos e filtros retornam todos os registros sem limite. Problema de memória e banda em produção.
22	CPF sem validação de dígito verificador	Validado só no formato via @Pattern, sem checar os dígitos verificadores do algoritmo do CPF.
23	Sem versionamento, Swagger ou soft delete	Endpoints sem versão (/api/v1/...), sem documentação OpenAPI, e exclusões físicas sem possibilidade de auditoria ou recuperação.
24	Token de recuperação salvo em texto plano	UUID é seguro o bastante para o protótipo, mas salvar o hash do token seria a abordagem correta para produção.
25	Sem rate limiting em login e recuperação de senha	Endpoints /login e /recuperar-senha sem proteção contra força bruta ou abuso.

Priorizar os itens Críticos e Importantes antes do próximo deploy em produção. Melhorias futuras podem ser planejadas em sprints subsequentes.